GUIA DE DESENVOLVIMENTO FULLSTACK

Sistema de Gerenciamento de Clientes, Veículos e Agendamentos

1. Introdução

O projeto consiste no planejamento e desenvolvimento de um sistema Fullstack para o gerenciamento de clientes, veículos e agendamentos de serviços. A aplicação permitirá cadastrar clientes, associar veículos aos seus proprietários, organizar agendamentos e acompanhar os serviços programados.

O sistema contará com autenticação de usuários e mecanismos de segurança para proteger as informações armazenadas.

O desenvolvimento será dividido em etapas, desde a identificação dos requisitos até a publicação e manutenção da aplicação.

2. Objetivo do sistema

Desenvolver uma aplicação que facilite a organização dos dados de clientes e veículos, além do controle dos agendamentos de serviços. O sistema deverá permitir o acesso autorizado às funcionalidades, manter as informações organizadas e reduzir erros no gerenciamento das atividades.

3. Requisitos do sistema

3.1. Requisitos funcionais

Os requisitos funcionais representam as funcionalidades que o sistema deverá oferecer.

- RF01: Permitir o cadastro de usuários.
- RF02: Permitir o login e a autenticação dos usuários.
- RF03: Permitir o cadastro de clientes.
- RF04: Permitir a consulta, a edição e a exclusão de clientes.
- RF05: Permitir o cadastro de veículos.
- RF06: Permitir a associação de veículos aos respectivos clientes.
- RF07: Permitir a consulta, a edição e a exclusão de veículos, respeitando as dependências existentes.

- RF08: Permitir o cadastro de agendamentos de serviços.
- RF09: Permitir a consulta, a edição e o cancelamento de agendamentos.
- RF10: Permitir a consulta dos agendamentos por data, cliente ou veículo.
- RF11: Permitir o acompanhamento do status dos agendamentos.
- RF12: Restringir o acesso às funcionalidades e aos dados conforme as permissões do usuário.

3.2. Requisitos não funcionais

Os requisitos não funcionais definem as características de qualidade e segurança esperadas do sistema.

- RNF01: A interface deverá ser organizada e fácil de utilizar.
- RNF02: As senhas deverão ser armazenadas utilizando hash seguro.
- RNF03: O sistema deverá validar os dados recebidos pelo Backend.
- RNF04: O banco de dados deverá manter a integridade das informações.
- RNF05: As funcionalidades protegidas deverão exigir autenticação.
- RNF06: O sistema deverá impedir acessos não autorizados aos dados.
- RNF07: A aplicação deverá apresentar mensagens claras de sucesso e erro.
- RNF08: O projeto deverá utilizar controle de versão.
- RNF09: O sistema deverá permitir a realização de testes antes da publicação.
- RNF10: As credenciais e informações sensíveis deverão ser protegidas.

4. Tecnologias previstas

Frontend: React e TypeScript, para a construção da interface e das funcionalidades visuais.

Backend: Node.js e Express, para implementar a API, processar as requisições e executar as regras de negócio.

Banco de Dados: PostgreSQL, para armazenar usuários, clientes, veículos e agendamentos.

ORM: Prisma 7, para organizar o acesso ao banco de dados e gerenciar o modelo de dados.

Versionamento: Git e GitHub, para registrar as alterações e manter o código organizado.

Testes: Thunder Client para testar as rotas da API e ferramentas de testes automatizados quando necessário.

5. Planejamento do banco de dados

O banco de dados será organizado em entidades relacionadas entre si.

5.1. Usuário

Representa as pessoas autorizadas a acessar o sistema.

Informações previstas: identificador, nome, e-mail, hash da senha, status e data de cadastro.

5.2. Cliente

Representa a pessoa que utiliza os serviços oferecidos pelo estabelecimento.

Informações previstas: identificador, nome, CPF, telefone, e-mail e data de cadastro.

5.3. Veículo

Representa o veículo cadastrado no sistema.

Informações previstas: identificador, placa, marca, modelo, ano, cor e referência ao cliente proprietário.

5.4. Agendamento

Representa um serviço programado para determinado cliente e veículo.

Informações previstas: identificador, cliente, veículo, descrição do serviço, data, horário, status e data de criação.

O modelo definitivo deverá definir os tipos de dados, as chaves primárias, as chaves estrangeiras e as restrições necessárias para garantir a integridade dos registros.

6. Planejamento do Backend e da API

O Backend será responsável por receber as requisições do Frontend, validar os dados, verificar as permissões de acesso e realizar as operações necessárias no banco de dados.

A API será organizada por recursos, utilizando métodos HTTP adequados.

As principais funcionalidades serão:

- Autenticação de usuários.
- Gerenciamento de clientes.
- Gerenciamento de veículos.
- Gerenciamento de agendamentos.
- Consulta e filtragem de registros.
- Atualização dos dados e dos status dos agendamentos.

As rotas protegidas deverão verificar a identidade e as permissões do usuário antes de permitir operações sobre os dados.

7. Planejamento do Frontend

O Frontend será responsável pela interface utilizada pelos usuários.

As principais telas previstas são:

- Tela de login.
- Painel principal.
- Tela de cadastro e listagem de clientes.
- Tela de cadastro e listagem de veículos.
- Tela de detalhes do cliente e de seus veículos.
- Tela de criação de agendamentos.
- Tela de consulta dos agendamentos.
- Tela de edição e cancelamento de agendamentos.

A interface deverá apresentar formulários organizados, campos validados, mensagens de confirmação e recursos de pesquisa e filtragem.

8. Regras de negócio

O sistema deverá respeitar as seguintes regras:

- O e-mail de cada usuário deverá ser único.
- Cada veículo deverá possuir uma identificação adequada, incluindo a placa.
- Um veículo deverá estar associado a um cliente cadastrado.
- Um agendamento deverá estar associado a um cliente e a um veículo válido.
- A data e o horário do agendamento deverão ser válidos.
- O sistema deverá impedir conflitos de horários quando houver uma agenda compartilhada de serviços.
- Um agendamento cancelado não deverá ser tratado como um agendamento ativo.
- Somente usuários autenticados e autorizados poderão executar operações protegidas.
- As exclusões deverão respeitar os relacionamentos entre clientes, veículos e agendamentos.
- Os dados deverão ser validados no Backend, mesmo que também exista validação no Frontend.

9. Integração entre Frontend, Backend e Banco de Dados

A integração ocorrerá por meio da API.

Quando um usuário cadastrar um cliente, por exemplo, o Frontend enviará os dados ao Backend. O Backend verificará as informações, aplicará as regras de negócio e solicitará a gravação no PostgreSQL.

O resultado será enviado ao Frontend, que apresentará uma mensagem ao usuário e atualizará a listagem.

O mesmo processo será aplicado ao cadastro de veículos e ao gerenciamento de agendamentos.

10. Segurança

A segurança será aplicada em todas as etapas do desenvolvimento.

O sistema deverá implementar autenticação, autorização, validação de entradas e proteção das credenciais. As senhas serão armazenadas com hash seguro, e as operações sobre os registros deverão verificar as permissões do usuário.

As consultas ao banco deverão utilizar mecanismos seguros contra SQL Injection. As credenciais de conexão deverão ficar em variáveis de ambiente, fora do código-fonte versionado.

Também deverão ser consideradas a proteção das sessões ou dos tokens de autenticação, a configuração de HTTPS em produção e a minimização da exposição de dados pessoais.

11. Testes

Os testes deverão verificar o funcionamento das principais funcionalidades.

Serão testados:

- Cadastro e login de usuários.
- Tentativas de login com credenciais inválidas.
- Cadastro, consulta, edição e exclusão de clientes.
- Cadastro e gerenciamento de veículos.
- Associação entre veículos e clientes.
- Criação, edição e cancelamento de agendamentos.
- Tentativas de agendar horários conflitantes.
- Acesso a rotas sem autenticação.
- Tentativas de acessar dados sem permissão.
- Validação de campos obrigatórios e formatos inválidos.

Os testes de integração deverão verificar se o Frontend, a API e o banco de dados funcionam corretamente em conjunto.

12. Git e versionamento

O Git será utilizado para controlar o histórico de alterações. O projeto poderá ser armazenado no GitHub, permitindo acompanhar o desenvolvimento e manter uma cópia remota do código.

Os commits deverão apresentar mensagens claras sobre as alterações realizadas. Arquivos com senhas, tokens ou credenciais não deverão ser enviados ao repositório.

13. Deploy

Após a conclusão dos testes, o sistema poderá ser publicado em um ambiente de produção.

O processo envolverá a hospedagem do Frontend, a publicação do Backend e a configuração do PostgreSQL em um ambiente apropriado.

Também deverão ser configurados os endereços da API, as variáveis de ambiente, o HTTPS e as permissões de acesso.

Antes da publicação, deverão ser verificadas as configurações de segurança e o funcionamento das principais funcionalidades.

14. Manutenção e evolução

Após a publicação, o sistema deverá ser monitorado para identificar falhas e corrigir problemas.

A manutenção poderá incluir atualização de dependências, correção de erros, backups, monitoramento de desempenho e melhorias na interface.

Futuramente, poderão ser adicionadas funcionalidades como notificações de agendamento, calendário de serviços, histórico de atendimentos, relatórios e diferentes níveis de permissão para os usuários.

15. Etapas de desenvolvimento

O projeto seguirá esta sequência:

1. Entendimento do problema e definição dos requisitos.
2. Planejamento da arquitetura e das funcionalidades.
3. Modelagem do banco de dados.
4. Configuração do ambiente de desenvolvimento.
5. Criação e configuração do banco de dados.
6. Desenvolvimento do Backend e da API.
7. Implementação das regras de negócio e da autenticação.
8. Desenvolvimento do Frontend.
9. Integração entre Frontend, Backend e Banco de Dados.
10. Verificação das medidas de segurança.
11. Realização dos testes.
12. Versionamento e organização do repositório.
13. Publicação do sistema.
14. Manutenção e evolução.

Essa ordem permite construir o sistema de maneira organizada, aproveitando o resultado de cada etapa na seguinte. Os testes, a segurança e o versionamento deverão acompanhar todo o

desenvolvimento.

16. Fontes de referência

React — Documentação oficial

https://react.dev/learn

Referência para a construção de interfaces e componentes.

Node.js — Documentação oficial

https://nodejs.org/docs/latest/api/

Referência para o ambiente de execução utilizado no Backend.

Express — Documentação oficial

https://expressjs.com/

Referência para a criação de servidores e APIs.

PostgreSQL — Documentação oficial

https://www.postgresql.org/docs/

Referência para a criação e o gerenciamento do banco de dados.

Prisma — Documentação oficial

https://www.prisma.io/docs

Referência para o acesso ao banco de dados e o gerenciamento do modelo de dados.

Git — Documentação oficial

https://git-scm.com/doc

Referência para o controle de versão.

OWASP — Authentication Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

Referência para boas práticas de autenticação.

OWASP — SQL Injection Prevention Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

Referência para a prevenção de ataques de SQL Injection.

17. Conclusão

O desenvolvimento do sistema de gerenciamento de clientes, veículos e agendamentos exige planejamento, organização dos dados, implementação de regras de negócio e integração entre Frontend, Backend e Banco de Dados.

A documentação estabelece uma base para orientar a construção da aplicação, garantindo que as funcionalidades sejam desenvolvidas de acordo com os requisitos e que os dados sejam tratados de maneira segura.

Seguindo as etapas propostas, será possível construir um sistema organizado, funcional e preparado para futuras melhorias.